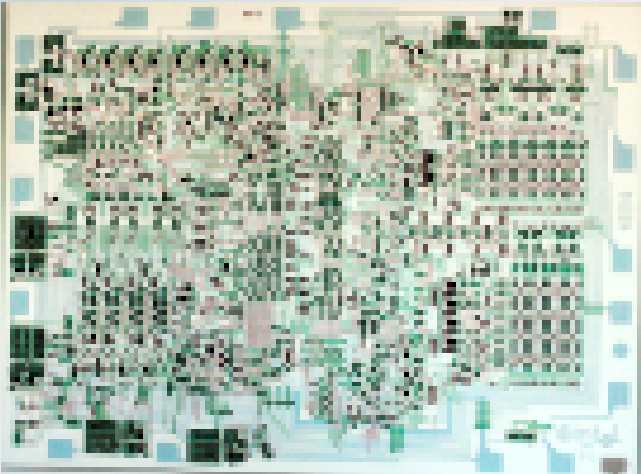




Lecture 9a: The TRAP Mechanism

CSCI11: Computer Architecture and Organization

Readings

- *Chapter 9* [Patt and Patel: Introduction to Computing Systems...](#)
- *Appendix A* [Patt and Patel: Introduction to Computing Systems...](#) - *Instructor Repo*
- *LC3 Cookbook* - *Instructor Repo*

Syllabus Updates

Date	Week	Content
3/30	8	RPN and SMC
4/5	9	TRAPS and I/O
4/12	10	Test 2

Final Project

Simple Math Calculator:

- Finished working program
- Complete repository on Github
- 15 minute discussion with Instructor

Program details in *Simple Math Calculator* pdf

LC-3 Memory Map

Address Range	Region	Access
x0000 – x00FF	Trap Vector Table	Supervisor
x0200 – x2FFF	OS / Supervisor region	Supervisor
x3000 – xFDFF	User programs	User + Supervisor
xFE00 – xFFFF	Device registers	Supervisor only

- Device registers sit at the top of the address space, gated behind *privilege*.
- The only legitimate way for user code to access them is through a TRAP.

Privilege

- **Separation of user and system code**
- Defines system programmers vs application programmers
- Establishes the idea of an Operating System

Why Separate OS and User Code?

The OS manages shared resources that every program depends on:

- **I/O devices** — keyboard, display, storage
- **Memory** — who owns which addresses
- **The CPU itself** — scheduling, exceptions, interrupts

Without separation, any program could:

- Overwrite OS code in memory — crashing the entire machine
- Read another program's data — violating privacy
- Drive hardware directly — causing device conflicts or data corruption
- Escalate its own privileges — bypassing all security

Real example:

MS-DOS (1980s) had no memory protection. A single buggy program could corrupt the OS in RAM and require a hard reboot. Every modern OS — Windows, macOS, Linux — enforces this boundary in hardware.

Remember "*Siberian Reset*"? The workstation would crash due to application code bugs, not because of system code bugs

Why Separation Requires Hardware Enforcement

Software-only separation can always be bypassed by software.

The boundary is only meaningful if the **CPU itself** refuses to execute certain operations unless it is in the right mode. This is the job of the **privilege bit**.

Two modes:

Mode	Who runs here	What it can do
User	Your programs	Normal computation, call OS via TRAP
Supervisor	The OS	Everything — devices, all memory, RTI

The CPU checks the privilege bit on *every* memory access and *every* restricted instruction. There is no software workaround.

Privilege: Why It Matters

In **supervisor mode** (PSR bit 15 = 0):

- All instructions are legal, including RTI
- Full access to all memory — including device registers at xFE00–xFFFF

In **user mode** (PSR bit 15 = 1):

- RTI raises a privilege violation exception
- Accessing device registers at xFE00–xFFFF raises an access violation

A **TRAP** is the *one sanctioned doorway* from user mode into the OS. The hardware controls entry: it switches the privilege bit, switches the stack, and jumps to an OS-supplied address — not an address the user program chose.

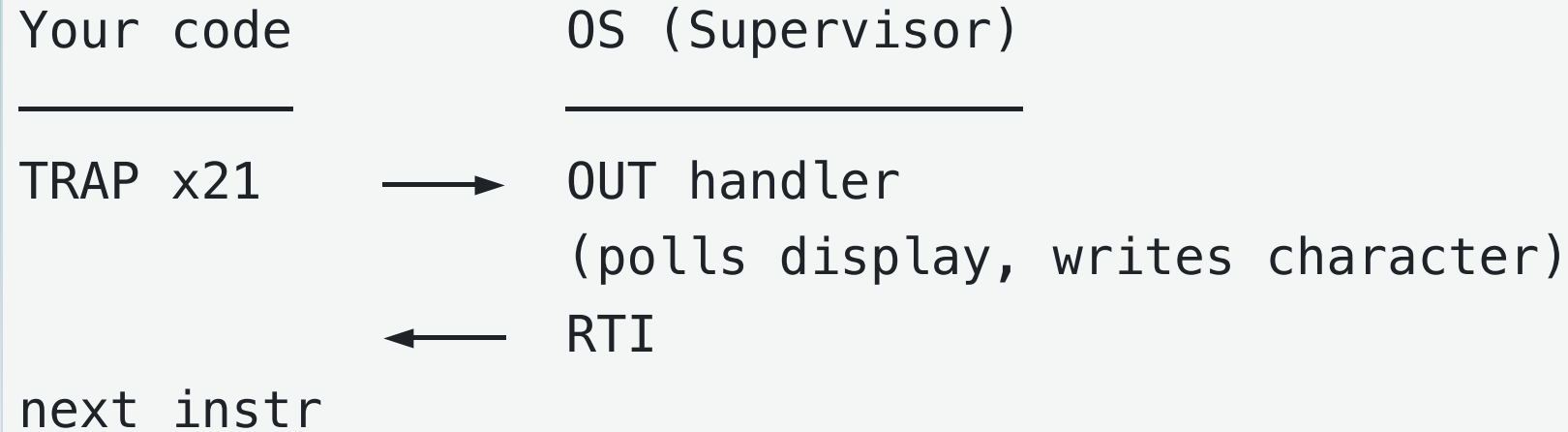
You Have Already Used Traps

Mnemonic	TRAP #	Effect
GETC	x20	Read character from keyboard (no echo)
OUT	x21	Write character in R0 to display
PUTS	x22	Write null-terminated string
IN	x23	Read character (with echo and prompt)
HALT	x25	Stop the machine

These are **OS service routines** — code that lives in the LC-3 operating system, not in your program.

How does your code jump into the OS and come back? That is what TRAP does.

TRAP: The Big Picture



1. Your program issues TRAP xnn
2. Hardware saves your return address and processor state
3. Execution jumps into the OS handler
4. RTI restores your state and returns

The mechanism that makes this safe and reliable is the **stack**.

TRAP Instruction Format



The 8-bit `trapvect8` is zero-extended to 16 bits and used to index the **Trap Vector Table** at `x0000–x00FF`.

TRAP `x21` → look up `M[x0021]` → that address holds the start of the OUT handler.

Traps allow the LC3 code to be extensible or expandable without having to add more opcodes (at the expense of readability)

The Trap Vector Table

Address	Contents	Trap
x0020	address of GETC handler	
x0021	address of OUT handler	
x0022	address of PUTS handler	
x0023	address of IN handler	
x0025	address of HALT handler	
x0026	<i>(user-defined — see Lec 9b)</i>	
x0027	<i>(user-defined — see Lec 9b)</i>	

The table is a jump table — each slot holds the *address* of a handler, not the handler code itself.

TRAP Execution — Step by Step

When the LC-3 executes `TRAP xnn`:

1. **Save PC** — push the address of the next instruction onto the supervisor stack
2. **Save PSR** — push the Processor Status Register onto the supervisor stack
3. **Set supervisor mode** — clear bit 15 of PSR (user → supervisor)
4. **Load new PC** — $PC \leftarrow M[x00nn]$ (from *Trap Vector Table*)
5. Execution continues in the OS handler

When the handler executes `RTI`:

6. **Restore PSR** — pop from supervisor stack (restores privilege and N/Z/P)
7. **Restore PC** — pop from supervisor stack → return to user program

What Is the PSR?

Processor Status Register (address xFFFC):

```
Bit 15:    Privilege  (1 = user mode,  0 = supervisor mode)
Bits 10-8:  Priority level (0-7)
Bit 2:      N   (negative)
Bit 1:      Z   (zero)
Bit 0:      P   (positive)
```

The N/Z/P bits you already know.

The **privilege bit** controls what memory the CPU can access and what instructions it can execute. This is the gate that protects the OS.

Two Stacks, One Register

R6 is the stack pointer in *both* modes — but it points to different stacks:

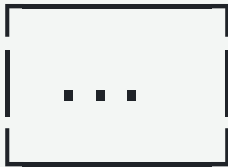
Mode	R6 points to
User (PSR bit 15 = 1)	User stack (e.g., x4000 downward)
Supervisor (PSR bit 15 = 0)	Supervisor stack (~x2FFF downward)

When TRAP switches from user to supervisor mode, the hardware automatically saves the user R6 (to x2FFE) and loads the supervisor R6. You do not write this swap — the hardware does it.

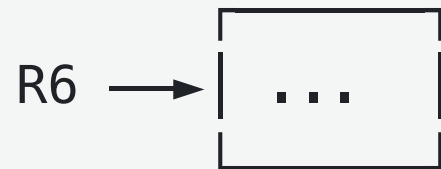
Stack State During a TRAP

Before TRAP

Supervisor Stack

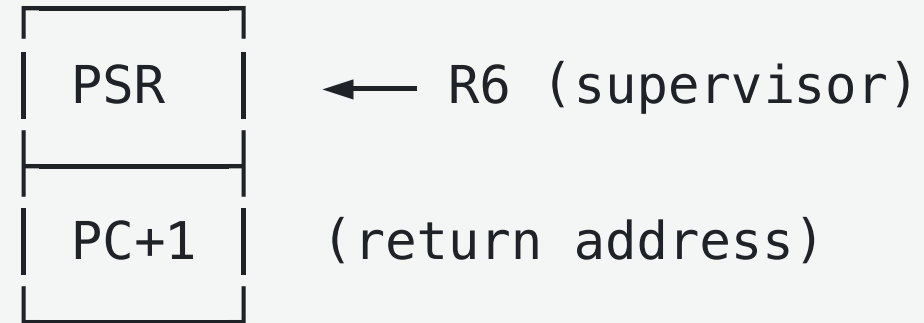


User Stack (unchanged)

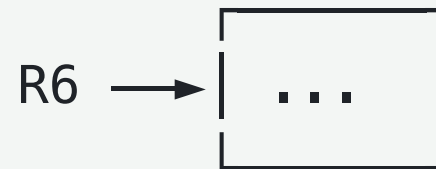


After TRAP xnn executes

Supervisor Stack



User Stack (unchanged)



PC+1 is the instruction *after* the TRAP — where execution resumes on RTI.

RET vs. RTI

Both return from a call — but they are *not* the same instruction.

	RET	RTI
Expands to	JMP R7	(no equivalent)
Returns from	Subroutine (JSR)	Trap or interrupt handler
Return address source	R7 (saved by JSR)	Supervisor stack (pushed by hardware)
Restores processor state?	No	Yes — pops PSR (privilege, priority, N/Z/P)
Mode required	User or supervisor	Supervisor only
Wrong mode	No restriction	Privilege violation exception

RET only moves the PC. RTI restores the full processor context.

The User Stack

Initializing R6 is **not required** just to call a TRAP.

When TRAP fires, the hardware automatically switches to the *supervisor* stack — it saves the current user R6 and loads the supervisor R6. The user R6 value does not affect this.

You only need to initialize R6 when your **user program** itself uses the stack — JSR subroutines, or your own PUSH/POP operations:

```
        .ORIG x3000
        LD R6, USR_STACK      ; required ONLY if user code uses the stack
        JSR SOME_SUBROUTINE  ; JSR saves R7; needs a valid R6
        ; ...
USR_STACK .FILL x4000
        .END
```

A User-Defined Trap Has Three Sections

Every custom trap needs *three separate .ORIG blocks*:

Section 1 – Vector Table Entry

```
.ORIG x00nn          ; slot in the trap vector table
.FILL TRAP_LABEL     ; address of your handler
.END
```

Section 2 – Handler Code

```
.ORIG x0300          ; somewhere in OS/supervisor space
TRAP_LABEL ...       ; actual code
RTI
.END
```

Section 3 – User Test / Example Code

```
.ORIG x3000          ; normal user program
LD R6, USR_STACK
TRAP xnn             ; call your trap
...
```

Example 1: `trap_chs` — Vector Table

The simplest possible trap. Takes R0, returns -R0.

Section 1: Vector Table

```
.ORIG x0026  
.FILL TRAP_CHS      ; x0026 → handler address  
.END
```

Example 1: `trap_chs` — Handler

Section 2: Handler

```
TRAP_CHS    .ORIG x0300
             NOT R0, R0
             ADD R0, R0, #1      ; two's complement negation
             RTI
             .END
```

No register saves needed — we only touch R0, which is the return value.

Example 1: `trap_chs` — Example User Code

Section 3: Test Code

```
        .ORIG x3000
        LD R6, USR_STACK      ; initialize stack pointer
        LD R0, NUMBER         ; R0 = 17
        TRAP x26              ; call CHS: R0 becomes -17
        ST R0, NUMBER         ; store result
        HALT

NUMBER   .FILL    #17
USR_STACK .FILL    x4000
        .END
```

Before: `NUMBER = 17`

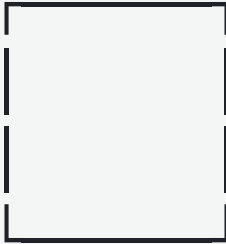
After: `NUMBER = -17 (xFFEF)`

Stack During `trap_chs`

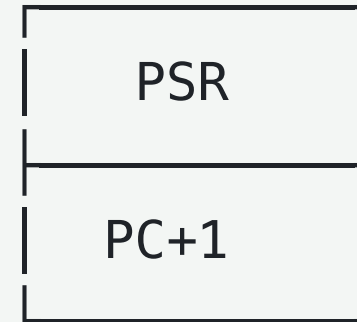
USR_STACK = x4000

User Stack (untouched)

x4000



Supervisor Stack



← R6 (sup)

CHS handler only uses R0 – no additional stack activity.
RTI pops PSR and PC, returns to user.

This is the minimum viable trap.

Why Save Registers?

`trap_chs` only needs R0 (the input/output register) — no saves required.

But what if the handler needs scratch registers?

```
; If we modify R1 inside the handler without saving it...  
TRAP_NEW      ADD R1, R0, #1      ; R1 is corrupted from user's view!  
              RTI
```

The user program has no idea their R1 was changed.

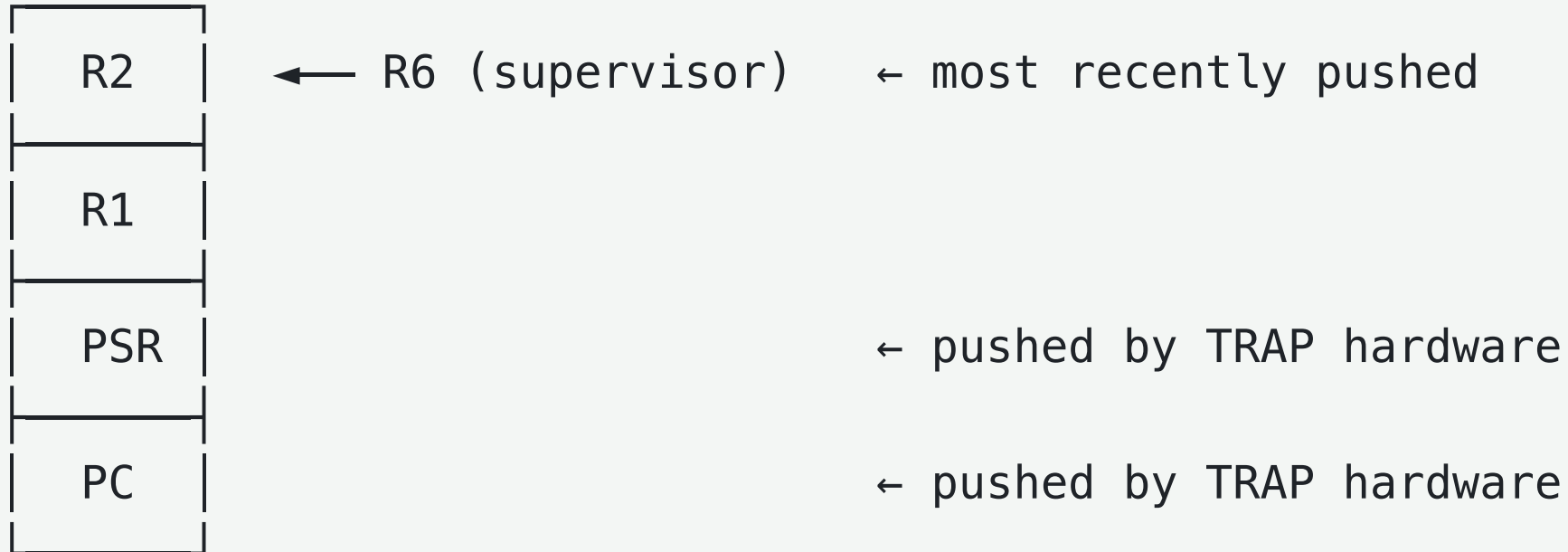
Rule: A trap handler must restore every register it modifies *except* R0 (the agreed return register).

```
; --- Save registers used ---  
ADD R6, R6, #-1  
STR R1, R6, #0      ; push R1  
ADD R6, R6, #-1  
STR R2, R6, #0      ; push R2  
; --- Do work ---  
; R1, R2 are free to use here  
; --- Restore in reverse order ---  
LDR R2, R6, #0  
ADD R6, R6, #1      ; pop R2  
LDR R1, R6, #0  
ADD R6, R6, #1      ; pop R1  
  
RTI
```

Last in, first out. Pop in the *reverse* order of push.

Stack State With Register Saves

Supervisor Stack after saving R1, R2



RTI requires PSR and PC to be on top at the moment it executes.
Your pops must exactly balance your pushes.

Designing a Trap: Checklist

Before you write a single instruction:

1. **Choose a vector** — pick an unused slot (x26, x27, ...)
2. **Define the interface** — what goes in, what comes out, which registers change
3. **Identify scratch registers** — which registers will the handler modify?
4. **Save/restore those registers** using the stack
5. **Place handler code** in supervisor space (x0300 or higher, avoid user area)
6. **End with RTI**, not HALT or BR

Common Trap Design Mistakes

Mistake	Consequence
Forget <code>LD R6, USR_STACK</code> in user code	<code>R6 = 0</code> ; first TRAP corrupts memory at <code>x0000</code>
Push 3 registers, pop 2	Stack misaligned; RTI returns to wrong address
Place handler at <code>x3000</code>	Overwrites user program; handler and test cannot coexist
Use RTI in user mode	Privilege violation exception
Forget vector table entry	TRAP <code>xnn</code> jumps to random memory

